

Fixed macros

Macro D is locked at (8, 4) on the top row, width two blocks sites eight and nine

The idea

- Fixed cells are placed first; occupied intervals block shelf and Abacus trials on that row
- Legality adds a macro check, any drift off the lock fails
- Movable cells must route around macro sites, not slide through them

Pseudocode

- Macro legalization adds one precondition to the Abacus sketch: place locked cells first
- Their site intervals become obstacles for every later try-place on that row
- Open this module's examples file and find the Pseudocode section
- That written sketch is what you implement on the implement track and what the browser

Algorithm sketch

- The legality line in the sketch is new too
- On this instance Abacus still finishes legal with displacement four

Algorithm sketch — try these

```
INPUT: positions, widths, fixed F (e.g. D@(8,4))
OUTPUT: legal pack; macros never move
place every f in F at locked (x,y)
run Abacus/Tetris on movables only
F intervals block try-place / left-pack
fail legality if any macro drifted
GOLDEN: D stays (8,4); Abacus disp=4; legal
```

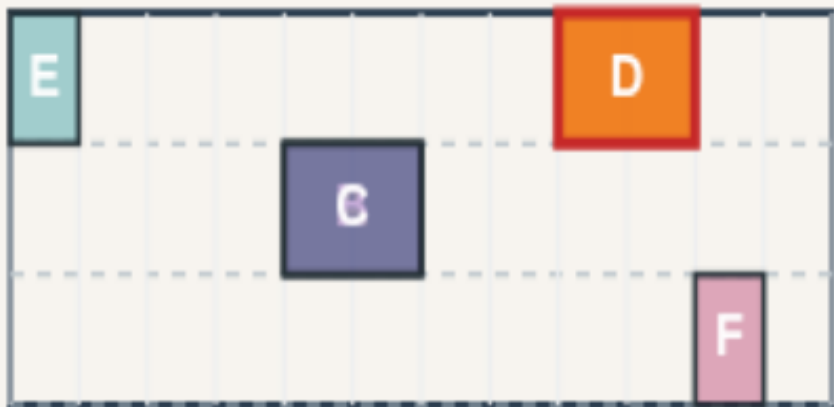
Macro D locked at (8, 4)

FIXED MACROS

Macro D locked at (8, 4)

Step 1 / 5 · d-locked · module03-01-fixed-macros

chip 12×6 · 3 rows · site 1



Explanation

Cell D is a fixed macro at (8, 4) on the top row—width two covers sites eight and nine. Every legalizer must leave D untouched while packing movables around it.

- `FIXED_MACROS.D = (8,4)`
- Width 2 blocks sites 8–9
- Movables: A,B,C,E,F

D fixed
top row y=4

Macro D locked at (8, 4)

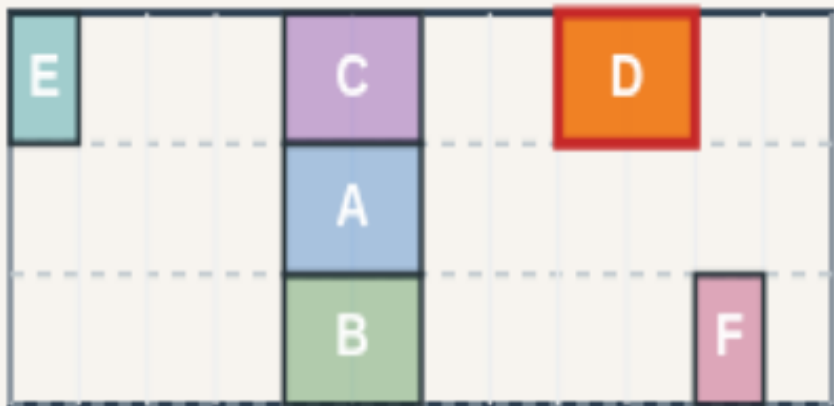
Abacus with fixed macros

FIXED MACROS

Abacus with fixed macros

Step 2 / 5 · abacus-fixed · module03-01-fixed-macros

chip 12x6 · 3 rows · site 1



Explanation

Run Abacus with the fixed map: D is placed first, occupied sites on row four block C from sliding into the macro. Movable cells trial rows respecting blocked intervals.

- `opts.fixed = FIXED_MACROS`
- Blocked intervals per row
- Pack around macro footprint

```
engine: abacusLegalize  
fixed: D
```

Abacus with fixed macros

D never moves

FIXED MACROS

D never moves

Step 3 / 5 · d-never-moves · module03-01-fixed-macros

chip 12×6 · 3 rows · site 1



Explanation

After Abacus with fixed macros, D remains at (8, 4).
Legality report includes a macro check—any drift off the lock fails the run.

- D @ (8,4) before and after
- Macro legality enforced
- disp still 4 on overlap seed

```
D.x=8 D.y=4  
disp: 4  
legal: true
```

D never moves

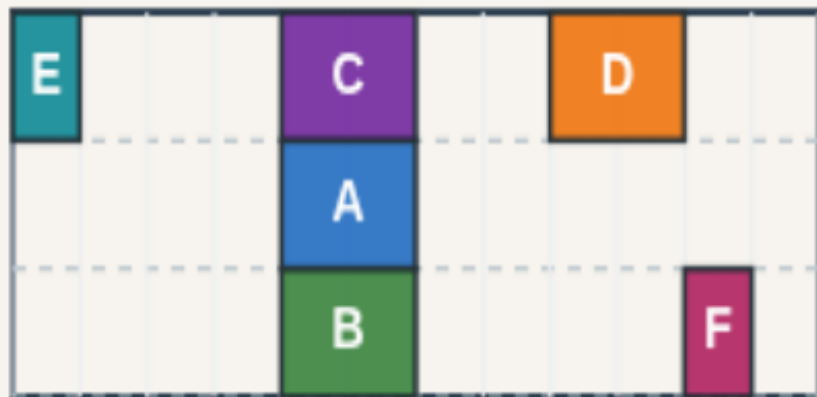
Still legal, displacement 4

FIXED MACROS

Still legal, displacement 4

Step 4 / 5 · still-legal · module03-01-fixed-macros

chip 12×6 · 3 rows · site 1



Explanation

With D fixed, Abacus still legalizes A, B, C, E, and F with total displacement four—the same as the unconstrained Abacus run on this instance because D never moved in either.

- Full legality: true
- No overlap with macro
- Movable avoid sites 8–9 on row 4

```
abacusDisp: 4
HPWL: 38
legal: true
```

Still legal, displacement 4

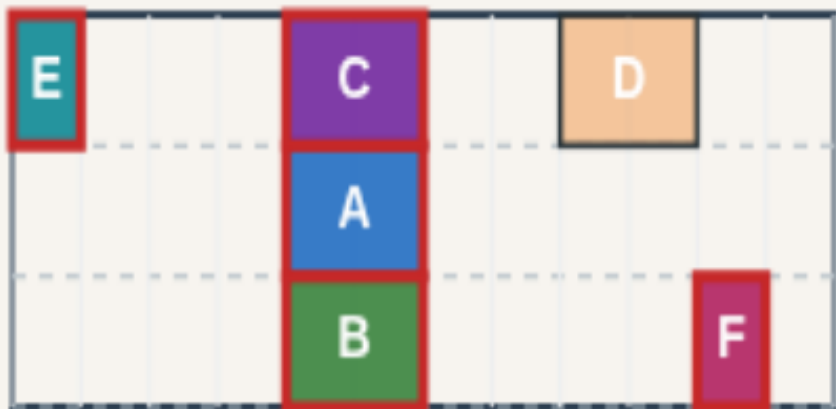
Movables avoid macro sites

FIXED MACROS

Movables avoid macro sites

Step 5 / 5 · avoid-macro · module03-01-fixed-macros

chip 12x6 · 3 rows · site 1



Explanation

C lands on row four at x four—left of E at zero, right of the macro gap. Packing algorithms must treat fixed macros as obstacles, not soft preferences.

- C @ (4,4) clears D
- E @ (0,4) on same row
- Never slide through macro

macro sites reserved
row-4 pack around D

Movables avoid macro sites

Browser lab track

- In the browser lab track, open the **fixed-macros** lab from the tools shelf
- Open the interactive lab
- Reveal golden is study-only
- Work the challenges that lock the goldens

Implement track

- In the implement track
- Parse ``tiny_legal.json``, run the algorithm with deterministic coordinates
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you're ready, take the short quiz, then continue to the next module